

Lab 03 — Praktijklab: leven, signalen en dood van een container

Doel: elk gedrag uit de cursus zelf uitlokken — de onmiddellijke stop, de tien seconden doodsstrijd, code 137, de automatische herstart — en zien wie er zonder daemon over je containers waakt.

Vereisten — Labs 01 en 02 afgewerkt. Images `alpine` en `nginx:alpine` aanwezig.

Geleverde bestanden — `files/demarrage-casse.sh` (kapot opstartscript) en `files/demarrage-correct.sh` (correct opstartscript), gebruikt in stap 3.

Stap 1 — De toestanden, één voor één

```
podman create --name toestand alpine sleep 120
podman ps -a --filter name=toestand --format 'table {{.Names}}\t{{.Status}}'
```

Observeer de status `Created`: de container bestaat, maar er draait nog geen enkel proces.

```
podman start toestand
podman ps --filter name=toestand --format '{{.Status}}'
podman pause toestand && podman ps -a --filter name=toestand --format '{{.Status}}'
podman unpause toestand && podman ps --filter name=toestand --format '{{.Status}}'
podman stop -t 2 toestand && podman ps -a --filter name=toestand --format '{{.Status}}'
podman start toestand && podman ps --filter name=toestand --format '{{.Status}}'
podman rm -f -t 0 toestand
```

Observeer achtereenvolgens `Up 1 second`, `Paused`, `Up 5 seconds`, de waarschuwing `StopSignal SIGTERM failed to stop container toestand in 2 seconds, resorting to SIGKILL`, `Exited (137)`, en dan opnieuw `Up`.

Uitleg. Een `Exited`-container kun je opnieuw starten: zijn configuratie en zijn schrijflaag zijn er nog. Merk ook op dat `podman ps` zonder `-a` een gepauzeerde container **niet toont**: die is niet "running".

Stap 2 — Waarom een container vanzelf stopt

```
podman run --name poging1 alpine
podman ps -a --filter name=poging1 --format '{{.Status}}'
```

Observeer `Exited (0)`: het standaardcommando van `alpine` is `/bin/sh`, en zonder invoer sluit die meteen af.

```
podman run -d --name poging2 nginx:alpine
podman ps --filter name=poging2 --format '{{.Status}}'
```

Observeer `Up`: nginx draait op de voorgrond.

```
podman run --rm alpine sh -c 'sleep 60 & echo "op de achtergrond gestart"'
```

Observeer dat het commando **onmiddellijk** terugkeert, hoewel er wel degelijk een `sleep 60` gestart werd.

Uitleg. De `&` koppelde `sleep` los; de shell voerde `echo` uit en sloot af. Zodra PID 1 dood was, werd de container vernietigd — en `sleep` verdween mee. Dit is dé klassieke fout in opstartscripts.

Kijk wie er over `poging2` waakt terwijl hij draait:

```
podman inspect --format '{{.State.CommonPid}}' poging2
ps -o pid,ppid,user,comm -p $(podman inspect --format '{{.State.CommonPid}}' poging2)
```

Observeer een `conmon`-proces, onder **jouw** gebruiker: dat is de toezichthouder die Podman achter elke container laat staan — de enige "daemon" die je nog hebt, goed voor amper een paar honderd KB.

```
podman rm poging1 ; podman rm -f -t 0 poging2
```

Stap 3 — Het kapotte opstartscript, en de correctie

Kopieer de twee geleverde scripts:

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <pad-van-het-lab>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Voer het eerste script uit **in** een container, met de huidige map gemount:

```
podman run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
```

Observeer: de boodschap verschijnt en je krijgt meteen je prompt terug — de container is al dood en verwijderd.

```
podman run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
podman ps --filter name=correct --format '{{.Status}}'
podman top correct
```

Observeer dat de container `Up` blijft, en dat `podman top` `sleep 300` als PID 1 toont — zonder ouderproces `sh`.

Uitleg. De `exec` in het tweede script **verving** de shell door het eindcommando, dat zo PID 1 erft. Dat is precies wat de `exec`-vorm van een `ENTRYPOINT` doet; die komt in lab 04 aan bod.

→ LINUX

`exec` is een ingebouwd shellcommando dat de gelijknamige systeemaanroep gebruikt: het huidige proces laat zijn programma (de shell) vallen en laadt het gevraagde programma **op zijn plaats**, met behoud van zijn PID. Zonder `exec` maakt de shell een kindproces aan (`fork`) en wacht hij. Met `exec` is er helemaal geen shell meer.

```
podman rm -f -t 0 correct
```

Stap 4 — De 10 seconden doodsstrijd

Meet hoelang een stop duurt bij een proces dat `SIGTERM` negeert:

```
podman run -d --name wacht alpine sleep 300
time podman stop wacht
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' wacht
podman rm wacht
```

Observeer de waarschuwing `StopSignal SIGTERM failed to stop container wacht in 10 seconds, resorting to SIGKILL`, `real 0m10.1s` en `code=137 oom=false`.

Probeer opnieuw, nu met een mini-init:

```
podman run -d --init --name wacht alpine sleep 300
podman exec wacht ps -o pid,comm
time podman stop wacht
podman inspect --format 'code={{.State.ExitCode}}' wacht
podman rm wacht
```

Observeer `1 podman-init` met daaronder `sleep` als PID 2, een stop in `0m0.1s` en `code=143`.

En met een applicatie die haar signalen wél correct behandelt:

```
podman run -d --name web nginx:alpine
time podman stop web
podman inspect --format 'code={{.State.ExitCode}}' web
podman rm web
```

Observeer een onmiddellijke stop en `code=0`.

Uitleg. Drie gedragingen, drie oorzaken. `sleep` als PID 1 **negeert** `SIGTERM` (bescherming van de kernel): de engine wacht en doodt → `137`. Met `--init` is `sleep` geen PID 1 meer, dus geldt de standaardactie → `143`. nginx installeert een signaalhandler en sluit netjes af (met code `0`, omdat nginx ervoor kiest normaal te eindigen). Vermenigvuldig die tien seconden met het aantal containers dat je draait, en je weet waar de onverklaarbare wachttijd in je heruitrol vandaan komt.

Je kunt de respijtperiode inkorten — al verhelpt dat de oorzaak niet:

```
podman run -d --name wacht alpine sleep 300
time podman stop -t 2 wacht
podman rm wacht
```

Observeer `real 0m2.1s`, nog altijd met code `137`.

Stap 5 — Exitcodes lezen

```
podman run --rm alpine sh -c 'exit 0' ; echo "code=$?"
podman run --rm alpine sh -c 'exit 3' ; echo "code=$?"
podman run --rm alpine onbestaand-commando ; echo "code=$?"
```

Observeer `0`, `3`, en dan `127` samen met `Error: crun: executable file onbestaand-commando not found in $PATH`.

```
podman run -d --name gedood alpine sleep 300
podman kill gedood
podman inspect --format 'code={{.State.ExitCode}}' gedood
podman rm gedood
```

Observeer `137`, deze keer zonder wachten: `kill` kent geen respijtpriode.

Veroorzaak nu een echt geheugentekort:

```
podman run --name oom --memory=32m --memory-swap=32m alpine sh -c 'head -c 100m /dev/zero |
tail'
echo "code=$?"
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' oom
podman rm oom
```

Observeer `code=137`, maar nu met `oom=true`: dezelfde code, een andere oorzaak — en alleen `inspect` maakt het onderscheid.

Uitleg. Boven 128 wijst de code op een dood door signaal: `code - 128` geeft het signaalnummer. `127` daarentegen is een opstartfout: de applicatie is nooit begonnen.

Stap 6 — `exec` tegenover `attach`

```
podman run -d --name web nginx:alpine
podman exec web nginx -v
podman exec -it web sh
```

Typ in de shell die je krijgt:

```
ps -o pid,comm
exit
```

Observeer dat `nginx` PID 1 is, gevolgd door zijn *workers*, en dat jouw `sh` een andere PID heeft. Na het verlaten van de shell staat de container **nog altijd** op `Up`.

```
podman ps --filter name=web --format '{{.Status}}'
```

Uitleg. `exec` startte een **nieuw** proces in de namespaces van de container; dat afsluiten raakt PID 1 niet. `attach` daarentegen zou je aan `nginx` zelf koppelen — één `Ctrl+C` en hij stopt. Gebruik voor logs dus altijd:

```
podman logs --tail 5 web
podman logs -f --since 1m web      # Ctrl+C stopt hier alleen de weergave
```

Stap 7 — Logs komen alleen van `stdout`

```
podman run --rm --name logs-demo alpine sh -c \
  'echo "ik ga naar stdout"; echo "ik ga naar een bestand" > /tmp/app.log; sleep 1'
```

Observeer dat alleen de eerste regel verschijnt.

```
podman run -d --name logs-demo alpine sh -c \
  'echo "zichtbaar"; echo "onzichtbaar" > /tmp/app.log; sleep 120'
podman logs logs-demo
podman exec logs-demo cat /tmp/app.log
podman rm -f -t 0 logs-demo
```

Observeer dat `podman logs` alleen `zichtbaar` toont; de inhoud van het bestand krijg je enkel te zien door de container binnen te gaan.

Uitleg. `conmon` vangt alleen `stdout` en `stderr` van PID 1 op. Daarom moet een applicatie in een container naar de console loggen — en daarom stel je in een gecontaineriseerde Spring Boot geen `logging.file.name` in.

Stap 8 — Automatische herstart

```
podman run -d --restart=on-failure:3 --name onstabiel alpine \
  sh -c 'echo "start $(date +%T)"; sleep 3; exit 1'
sleep 20
podman ps -a --filter name=onstabiel --format '{{.Names}} {{.Status}}'
podman inspect --format 'herstarts={{.RestartCount}} code={{.State.ExitCode}}' onstabiel
podman logs onstabiel
```

Observeer een `RestartCount` van `3`, de status `Exited (1)`, en **vier** regels "start" in de logs: de eerste poging plus drie herstarts.

Uitleg. De logs van opeenvolgende uitvoeringen stapelen zich op in dezelfde container: de eerste regel bevat de oorspronkelijke oorzaak. `.State` daarentegen beschrijft alleen de **laatste** uitvoering.

```
podman rm onstabiel
podman events --since 2m --until 1s | grep onstabiel | awk '{print $5, $6}' | uniq -c
```

Observeer het logboek van gebeurtenissen: `container start`, `container died`, `container restart` ... Dit is de enige plek waar je de *geschiedenis* van een container ziet, niet alleen zijn huidige toestand.

Controleer tot slot de onverenigbaarheid die de cursus aankondigde:

```
podman run --rm -d --restart=always nginx:alpine
```

Observeer Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no".

PODMAN

En na een reboot? Test het zelf: `podman run -d --restart=always --name overlever nginx:alpine`, sluit **al** je Ubuntu-vensters en voer vanuit PowerShell `wsl --shutdown` uit. Open Ubuntu opnieuw: `podman ps` is leeg. Niemand heeft het beleid herlezen — er is geen daemon. Op een server neemt `systemd` die rol over via een Quadlet-bestand (lab 10). Op je eigen machine is dit prima: ontwikkelcontainers hoeven geen herstart te overleven. Ruim daarna op met `podman rm -f -t 0 overlever`.

Stap 9 — Bewijs uit een dode container halen

```
podman run --name autopsyie alpine sh -c 'echo "belangrijk spoor" > /rapport.txt; exit 2'
podman ps -a --filter name=autopsyie --format '{{.Status}}'
podman cp autopsyie:/rapport.txt ./rapport.txt
cat rapport.txt
```

Observeer dat je het bestand kunt recupereren terwijl de container op `Exited (2)` staat.

```
podman rm autopsyie
podman cp autopsyie:/rapport.txt ./ander.txt
```

Observeer Error: container "autopsyie" does not exist: is de container weg, dan is alles weg.

Uitleg. De gouden regel in beheer: **eerst inspecteren, dan pas verwijderen**. `cp`, `logs` en `inspect` werken op een gestopte container — nooit op een verwijderde.

Opruimen

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
podman rm -f -t 0 wacht web correct onstabiel autopsyie poging1 poging2 gedood toestand oom
overlever 2>/dev/null
rm -f ~/labo-docker/03/rapport.txt
podman ps -a --format '{{.Names}}'
```

Observeer dat er geen enkele container van dit lab overblijft. De images `alpine` en `nginx:alpine` blijven staan.

Wat je nu moet kunnen beweren

- Een container sterft samen met zijn PID 1 — je hebt alle drie de gevallen zelf uitgelokt.
- `sleep` als PID 1 negeert `SIGTERM`; `--init` verhelpt het symptoom, `exec` de oorzaak.
- `137` = gedood (door `stop`, `kill` of de OOM killer — `inspect` beslist), `143` = netjes gestopt, `127` = commando niet gevonden.

- `exec` start een proces, `attach` koppelt aan PID 1.
- `podman logs` toont alleen `stdout` / `stderr`, opgevangen door `conmon`.
- `podman rm` vernietigt logs en bewijsmateriaal: eerst inspecteren.
- Zonder daemon overleeft een *restart policy* geen `wsl --shutdown`.